Zion

Audit Report

contact@bitslab.xyz

https://twitter.com/movebit_

Thu Feb 13 2025

Zion Audit Report

1 Executive Summary

1.1 Project Information

Description	Zion is a crash game where players can use leveraged bets.
Type	Game
Auditors	MoveBit
Timeline	Mon Dec 23 2024 - Thu Feb 13 2025
Languages	Move
Platform	Movement
Methods	Architecture Review, Unit Testing, Manual Review
Source Code	https://github.com/SeamMoney/cash-markets
Commits	2b1f2a08027787d11a0095edba5dae979fbc3ad3 55db5684d81cc167d7b30a5cb0107d0917522a40 bff7f8f06457242db00e0b295d2e94e1541ea43d

1.2 Files in Scope

The following are the SHA1 hashes of the original reviewed files.

ID	File	SHA-1 Hash
MOV	move-mania-contracts/Move.toml	28a7af98309ae35c7f93393830aa15d14f5e176a
LPO	move-mania-contracts/sources/liquidity_pool.move	40370a259487e68ac370ffdaddfd243bf4a5e0dc
CRA	move-mania-contracts/sources/crash.move	38131dfe0becbfb30a7ae37ad40fd1ed481a146f
ZAP	move-mania-contracts/sources/z_apt.move	dd50243cd80850d0fc8b90da3de9835a7959665a

1.3 Issue Statistic

Item	Count	Fixed	Acknowledged
Total	15	14	1
Informational	3	2	1
Minor	4	4	0
Medium	2	2	0
Major	3	3	0
Critical	3	3	0

1.4 MoveBit Audit Breakdown

MoveBit aims to assess repositories for security-related issues, code quality, and compliance with specifications and best practices. Possible issues our team looked for included (but are not limited to):

- Transaction-ordering dependence
- Timestamp dependence
- Integer overflow/underflow by bit operations
- Number of rounding errors
- Denial of service / logical oversights
- Access control
- Centralization of power
- Business logic contradicting the specification
- Code clones, functionality duplication
- Gas usage
- Arbitrary token minting
- Unchecked CALL Return Values
- The flow of capability
- Witness Type

1.5 Methodology

The security team adopted the **"Testing and Automated Analysis"**, **"Code Review"** and **"Formal Verification"** strategy to perform a complete security test on the code in a way that is closest to the real attack. The main entrance and scope of security testing are stated in the conventions in the "Audit Objective", which can expand to contexts beyond the scope according to the actual testing needs. The main types of this security audit include:

(1) Testing and Automated Analysis

Items to check: state consistency / failure rollback / unit testing / value overflows / parameter verification / unhandled errors / boundary checking / coding specifications.

(2) Code Review

The code scope is illustrated in section 1.2.

(3) Formal Verification(Optional)

Perform formal verification for key functions with the Move Prover.

(4) Audit Process

- Carry out relevant security tests on the testnet or the mainnet;
- If there are any questions during the audit process, communicate with the code owner in time. The code owners should actively cooperate (this might include providing the latest stable source code, relevant deployment scripts or methods, transaction signature scripts, exchange docking schemes, etc.);
- The necessary information during the audit process will be well documented for both the audit team and the code owner in a timely manner.

2 Summary

This report has been commissioned by Zion to identify any potential issues and vulnerabilities in the source code of the Zion smart contract, as well as any contract dependencies that were not part of an officially recognized library. In this audit, we have utilized various techniques, including manual code review and static analysis, to identify potential vulnerabilities and security issues.

During the audit, we identified 15 issues of varying severity, listed below.

ID	Title	Severity	Status
CRA-1	`cash_out` Lacks Access Control	Critical	Fixed
CRA-2	Missing Crash Point Check in `cash_out` Function	Critical	Fixed
CRA-3	Potential Inflation Attack	Major	Fixed
CRA-4	No Restrictions On `LPCoinType`	Medium	Fixed
CRA-5	Lack of Refund Mechanism in `force_remove_game`	Medium	Fixed
CRA-6	Redundant Parameter in `place_bet` Function	Minor	Fixed
CRA-7	Test Code in the Contract	Informational	Fixed
CRA-8	Duplicate Event Emit	Informational	Acknowledged
CRA-9	Incorrect Comment	Informational	Fixed
LPO-1	`liquidity_pool` Liquidity Pool Arbitrage Vulnerability Due to Predictable Reserve Changes	Critical	Fixed

LPO-2	Logic Error In `remove_liquidity` Function	Major	Fixed
LPO-3	Lack of Permission Control	Major	Fixed
LPO-4	Missing Validation in `remove_liquidity` Function	Minor	Fixed
LPO-5	The `supply_fa_liquidity` Function Lacks Validation To Ensure `amount_lp_coins_to_mint` Is Greater Than Zero	Minor	Fixed
WHI-1	Lack of Events Emit	Minor	Fixed

3 Participant Process

Here are the relevant actors with their respective abilities within the Zion Smart Contract :

Admin

- Admin can call the `start_game` function to start a new game of crash by generating a house secret and salt, hashing them, and passing the hashes to the function.
- Admin can call the `force_remove_game` function to remove the current game forcibly.
- Admin can call the `cash_out` function on behalf of a player to allow them to cash out their bet in the current game of crash.
- Admin can call the `reveal_crashpoint_and_distribute_winnings` function to reveal the crash point and distribute winnings to the players based on their bets and cash-out points.

Player

- Players can call the `place_bet` function to place a bet in the current game of crash.
- Players can call the `cash_out` function to cash out their bet in the current game of crash.(Admin will trigger this on behalf of the player in the current implementation).

Liquidity Provider

- Liquidity Providers can call the `supply_liquidity` function to supply liquidity to the pool and receive LP tokens.
- Liquidity Providers can call the `remove_liquidity` function to remove liquidity from the pool by burning their LP tokens.
- Liquidity Providers can call the `lock_lp_coins` function to lock their LP tokens in the liquidity pool.

4 Findings

CRA-1 `cash_out` Lacks Access Control

Severity: Critical

Status: Fixed

Code Location:

move-mania-contracts/sources/crash.move#329

Descriptions:

The `cash_out` function contains critical security vulnerabilities. First, it includes an unused admin parameter that serves no purpose and creates misleading code structure. More importantly, the function accepts a player address as a parameter instead of using the transaction signer, allowing any user to modify another player's cash_out value. This design flaw creates a severe security risk where malicious actors can manipulate other players' bets. The function should be modified to use the transaction signer's address directly, ensuring only bet owners can cash out their own bets.

Suggestion:

It is recommended to modify the function's permission check.

Resolution:

This issue has been fixed. The client has adopted our suggestions.

CRA-2 Missing Crash Point Check in `cash_out` Function

Severity: Critical

Status: Fixed

Code Location:

move-mania-contracts/sources/crash.move#231

Descriptions:

The `cash_out` function lacks a critical validation check against the game's crash point. Currently, users can call `cash_out` even after the crash point has been revealed through `reveal_crashpoint`, allowing them to see the game's outcome before deciding to cash out. This creates a significant exploit where users can guarantee profits by only cashing out when they know their desired multiplier is below the crash point.

Suggestion:

It is recommended to check whether the crash point has already been revealed.

Resolution:

This issue has been fixed. The client has adopted our suggestions.

CRA-3 Potential Inflation Attack

Severity: Major

Status: Fixed

Code Location:

move-mania-contracts/sources/crash.move#239

Descriptions:

The `supply_liquidity` function does not enforce a check to ensure `amount_lp_coins_to_mint > 0`, allowing users to inject a large amount of `reserve_coin` when `lp_coin_supply` is 0. This could artificially inflate the LP token price and facilitate an inflation attack, diluting future liquidity providers.

Suggestion:

Add validation to ensure `amount_lp_coins_to_mint` > 0 before minting LP tokens.

Resolution:

This issue has been fixed. The client has adopted our suggestions.

CRA-4 No Restrictions On `LPCoinType`

Severity: Medium

Status: Fixed

Code Location:

move-mania-contracts/sources/crash.move#404

Descriptions:

Since the `distribute_winnings` function does not restrict the type of `LPCoinType`, a user may place a bet in the pool `<BettingCoinType-A>`, but choose pool `<BettingCoinType-B>` when settling. This may cause problems with inconsistent settlement data.

Suggestion:

It is recommended to make sure that this is in line with the protocol design and whether `BettingCoinType` is possible to combine with multiple LPCoin to a pool.

Resolution:

This issue has been fixed. The client has adopted our suggestions.

CRA-5 Lack of Refund Mechanism in `force_remove_game`

Severity: Medium

Status: Fixed

Code Location:

move-mania-contracts/sources/crash.move#184

Descriptions:

When an admin forcefully removes a game, all active bets are deleted without refunding the players' stakes. This could result in permanent loss of user funds locked in the contract.

Suggestion:

It is recommended to add a proper game close mechanism.

Resolution:

This issue has been fixed. The client has adopted our suggestions.

CRA-6 Redundant Parameter in `place_bet` Function

Severity: Minor

Status: Fixed

Code Location:

move-mania-contracts/sources/crash.move#231

Descriptions:

The `place_bet` function accepts a `token_addr_as_string` parameter that is never used in the function's logic.

Suggestion:

It is recommended to remove unnecessary parameters.

Resolution:

This issue has been fixed. The client has adopted our suggestions.

CRA-7 Test Code in the Contract

Severity: Informational

Status: Fixed

Code Location:

move-mania-contracts/sources/crash.move#153

Descriptions:

There is a test code in the `start_game` function;

```
print(&house_secret_hash);
```

Suggestion:

It is recommended that the test code `debug::print` be removed from the function.

Resolution:

This issue has been fixed. The client has adopted our suggestions.

CRA-8 Duplicate Event Emit

Severity: Informational

Status: Acknowledged

Code Location:

move-mania-contracts/sources/crash.move

Descriptions:

The `supply_liquidity` function emits the same `DepositEvent` twice: once using `event::emit_event` and again with `event::emit`. This redundancy can lead to unnecessary gas consumption and duplicate event logs, which may confuse downstream systems or users monitoring the blockchain.

Suggestion:

It is recommended to remove an event.

CRA-9 Incorrect Comment

Severity: Informational

Status: Fixed

Code Location:

move-mania-contracts/sources/crash.move#23

Descriptions:

The comment currently reads:

```
const MAX_CRASH_POINT: u128 = 340282366920938463463374607431768211455; // 2^64 - 1
```

The comment incorrectly states that `MAX_CRASH_POINT` is 2^64, while the actual value is 2^128-1.

Suggestion:

Update the comment to reflect the action being performed accurately.

Resolution:

This issue has been fixed. The client has adopted our suggestions.

LPO-1 `liquidity_pool` Liquidity Pool Arbitrage Vulnerability Due to Predictable Reserve Changes

Severity: Critical

Status: Fixed

Code Location:

move-mania-contracts/sources/liquidity_pool.move

Descriptions:

The `liquidity_pool` module's design allows liquidity providers to exploit predictable changes in the pool's reserves for risk-free profits. Since users will continuously add reserve tokens through the crash module's betting activities, liquidity providers can strategically time their deposits and withdrawals around these known reserve increases. This creates an arbitrage opportunity where providers can add liquidity before bets are placed and remove it after the reserve increases, guaranteeing profits without risk. This undermines the intended risk-sharing mechanism of the liquidity pool and could lead to unfair profit extraction at the expense of the betting system's sustainability.

Suggestion:

It is recommended to ensure that this complies with the protocol design.

Resolution:

This issue has been fixed. The client has adopted our suggestions.

LPO-2 Logic Error In `remove_liquidity` Function

Severity: Major

Status: Fixed

Code Location:

move-mania-contracts/sources/liquidity_pool.move#411

Descriptions:

When `maintaining_lp` is zero, the code still transfers the NFT back to the user (because the judgment condition is that the original `lp_amount > 0`), resulting in the user holding an invalid NFT (`lp_amount` is zero). It should be checked that `maintaining_lp > 0` before allowing the transfer.

Suggestion:

It's recommended to fix this issue.

Resolution:

This issue has been fixed. The client has adopted our suggestions.

LPO-3 Lack of Permission Control

Severity: Major

Status: Fixed

Code Location:

move-mania-contracts/sources/liquidity_pool.move#128

Descriptions:

The `init_fa_LP_pool` function lacks proper access control, allowing any user to create an LP pool arbitrarily.

Suggestion:

It is recommended to add a permission check to restrict access to trusted roles, such as the contract owner or an administrator.

Resolution:

This issue has been fixed. The client has adopted our suggestions.

LPO-4 Missing Validation in `remove_liquidity` Function

Severity: Minor

Status: Fixed

Code Location:

move-mania-contracts/sources/liquidity_pool.move#351

Descriptions:

This function does not check whether `amount_to_withdraw` is greater than 0, which could lead to unintended behavior, such as emitting unnecessary events or performing redundant calculations.

Suggestion:

It is recommended to add an assertion at the beginning to ensure `amount_to_withdraw > 0`.

Resolution:

This issue has been fixed. The client has adopted our suggestions.

LPO-5 The `supply_fa_liquidity` Function Lacks Validation To Ensure `amount_lp_coins_to_mint` Is Greater Than Zero

Severity: Minor

Status: Fixed

Code Location:

move-mania-contracts/sources/liquidity_pool.move#288

Descriptions:

The `supply_fa_liquidity` function lacks validation to ensure `amount_lp_coins_to_mint` is greater than zero.

Suggestion:

It is recommended to check whether `amount_lp_coins_to_mint` is greater than 0.

Resolution:

This issue has been fixed. The client has adopted our suggestions.

WHI-1 Lack of Events Emit

Severity: Minor

Status: Fixed

Code Location:

move-mania-contracts/sources/whitelist.move

Descriptions:

The contract lacks appropriate events for some key functions such as: `add_to_whitelist()`, and `remove_from_whitelist()`. The lack of event records for these functions may cause inconvenience in the subsequent tracking and contract status changes.

Suggestion:

It is recommended to emit events for these functions.

Resolution:

This issue has been fixed. The client has adopted our suggestions.

Appendix 1

Issue Level

- **Informational** issues are often recommendations to improve the style of the code or to optimize code that does not affect the overall functionality.
- **Minor** issues are general suggestions relevant to best practices and readability. They don't post any direct risk. Developers are encouraged to fix them.
- **Medium** issues are non-exploitable problems and not security vulnerabilities. They should be fixed unless there is a specific reason not to.
- **Major** issues are security vulnerabilities. They put a portion of users' sensitive information at risk, and often are not directly exploitable. All major issues should be fixed.
- **Critical** issues are directly exploitable security vulnerabilities. They put users' sensitive information at risk. All critical issues should be fixed.

Issue Status

- **Fixed:** The issue has been resolved.
- **Partially Fixed:** The issue has been partially resolved.
- **Acknowledged:** The issue has been acknowledged by the code owner, and the code owner confirms it's as designed, and decides to keep it.

Appendix 2

Disclaimer

This report is based on the scope of materials and documents provided, with a limited review at the time provided. Results may not be complete and do not include all vulnerabilities. The review and this report are provided on an as-is, where-is, and as-available basis. You agree that your access and/or use, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your own risk. A report does not imply an endorsement of any particular project or team, nor does it guarantee its security. These reports should not be relied upon in any way by any third party, including for the purpose of making any decision to buy or sell products, services, or any other assets. TO THE FULLEST EXTENT PERMITTED BY LAW, WE DISCLAIM ALL WARRANTIES, EXPRESS OR IMPLIED, IN CONNECTION WITH THIS REPORT, ITS CONTENT, RELATED SERVICES AND PRODUCTS, AND YOUR USE, INCLUDING BUT NOT LIMITED TO THE IMPLIED WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, NOT INFRINGEMENT.